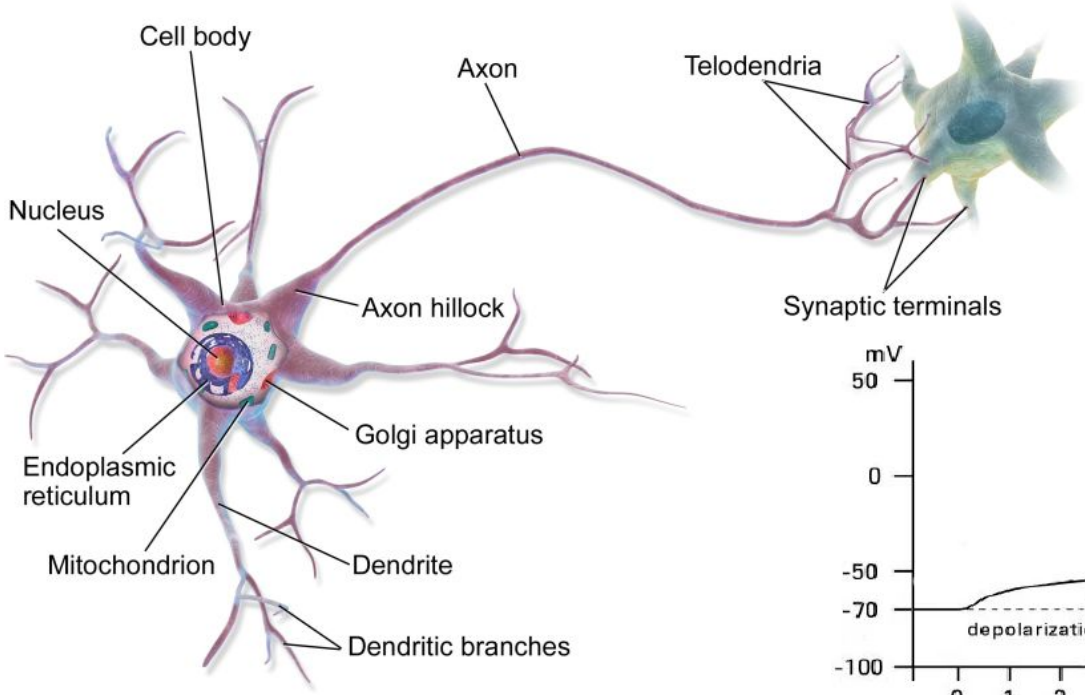
Neural Networks

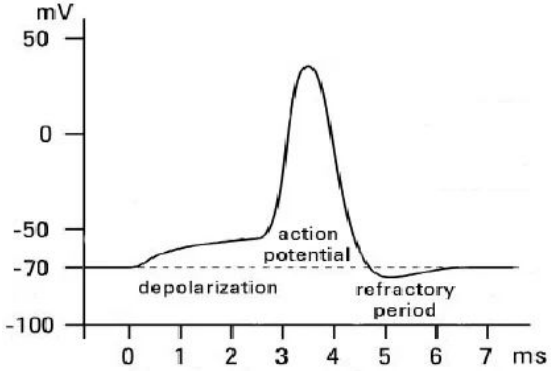
Tópicos em Fundamentos e Métodos da Computação

Prof. Dr. Luís Paulo Faina Garcia

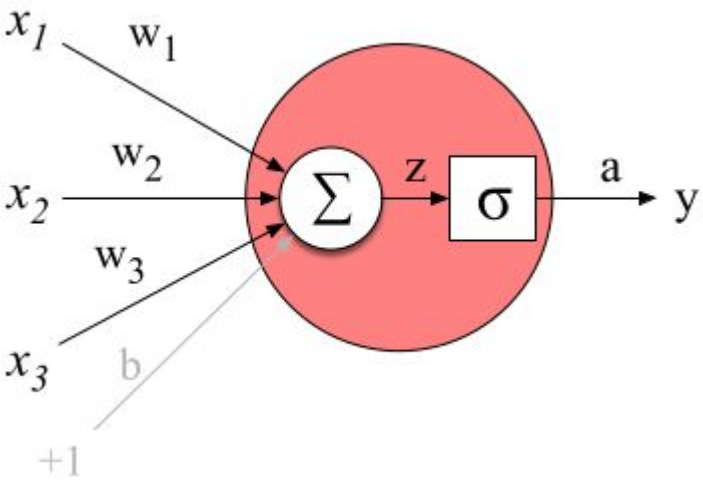
Universidade de Brasília
Departamento de Ciência da Computação

This is your brain

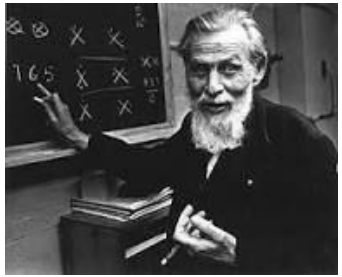




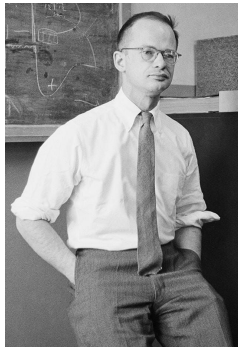
This is not in your brain



Neuron (1943)



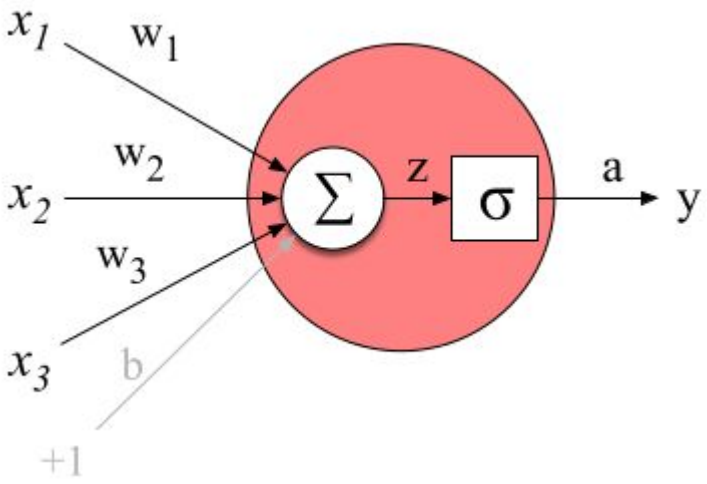
Warren McCulloch (Neurocientista)



Walter Pitts (matemático)

Warren McCulloch -Pitts neuron

This is not in your brain



Take weighted sum of inputs, plus bias:

$$z = b + \sum_i w_i x_i$$

$$z = w \cdot x + b$$

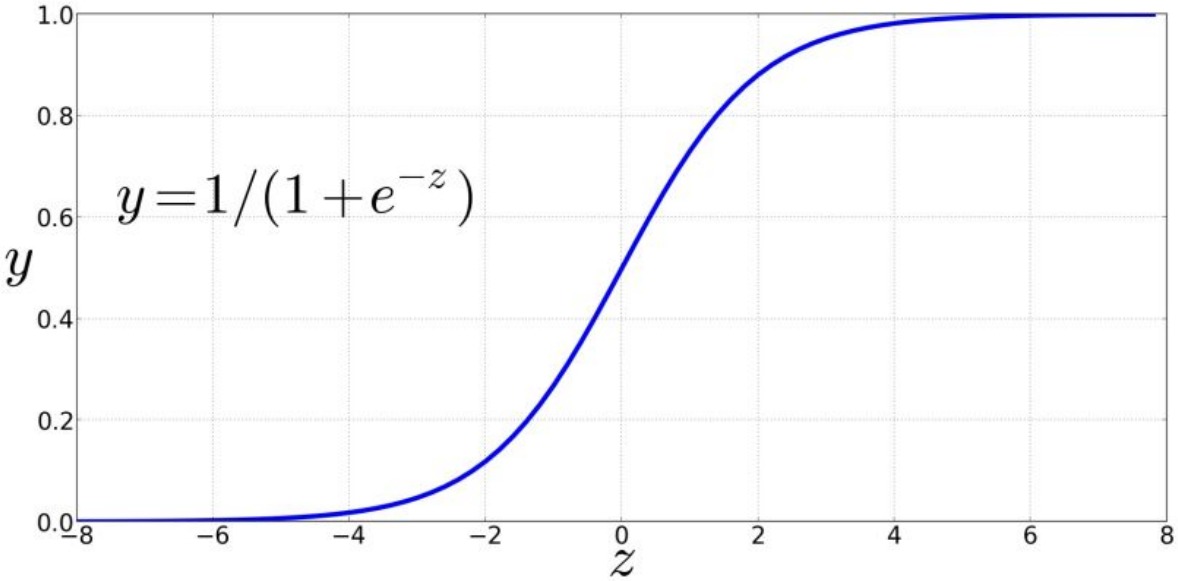
Applying a nonlinear activation function:

$$y = a = f(z)$$

Non-linear Activation Functions

Sigmoid

$$y = \sigma(z) = \frac{1}{1+e^{-z}}$$



The final function

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Example

Suppose:

```
w = [0.2, 0.3, 0.9]
```

```
b = 0.5
```

What happened with input x:

```
x = [0.5, 0.6, 0.1]
```

Example

Suppose:

$$y = \sigma(w \cdot x + b) =$$

```
w = [0.2, 0.3, 0.9]
```

```
b = 0.5
```

What happened with input x:

```
x = [0.5, 0.6, 0.1]
```

Example

Suppose:

```
w = [0.2, 0.3, 0.9]

b = 0.5
```

What happened with input x:

```
x = [0.5, 0.6, 0.1]
```

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

Example

Suppose:

```
w = [0.2, 0.3, 0.9]

b = 0.5
```

What happened with input x:

```
x = [0.5, 0.6, 0.1]
```

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

$$\frac{1}{1 + e^{-(.5*.2+.6*.3+.1*.9+.5)}} =$$

Example

Suppose:

```
w = [0.2, 0.3, 0.9]

b = 0.5
```

What happened with input x:

```
x = [0.5, 0.6, 0.1]
```

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

$$\frac{1}{1 + e^{-(.5*.2+.6*.3+.1*.9+.5)}} =$$

$$\frac{1}{1 + e^{-0.87}} =$$

Example

Suppose:

```
w = [0.2, 0.3, 0.9]

b = 0.5
```

What happened with input x:

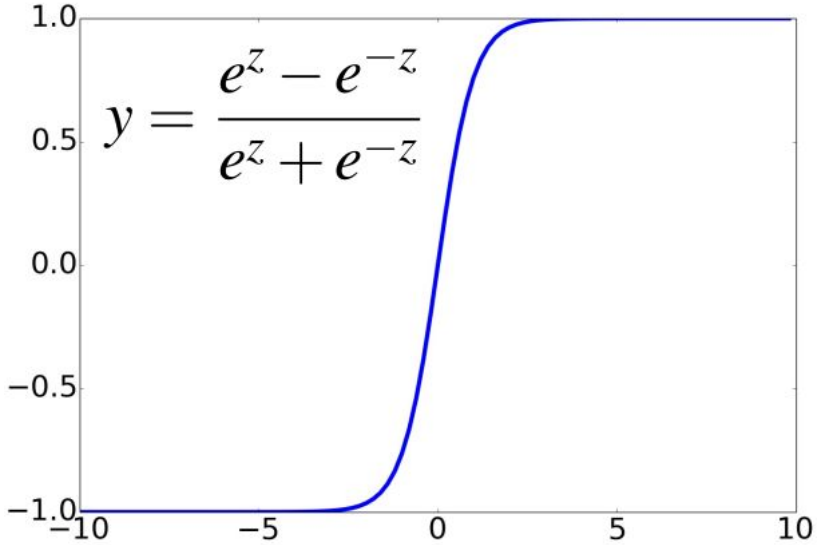
```
x = [0.5, 0.6, 0.1]
```

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

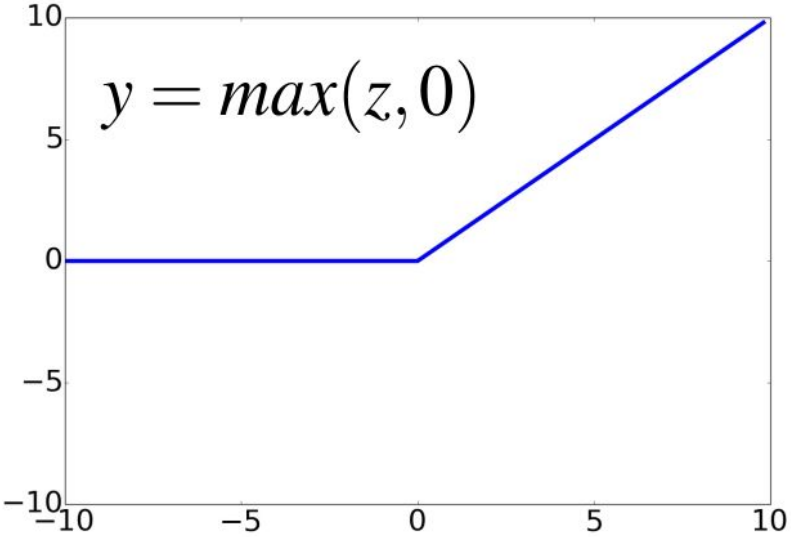
$$\frac{1}{1 + e^{-(.5*.2+.6*.3+.1*.9+.5)}} =$$

$$\frac{1}{1 + e^{-0.87}} = .70$$

Other Non-linear Activation Functions

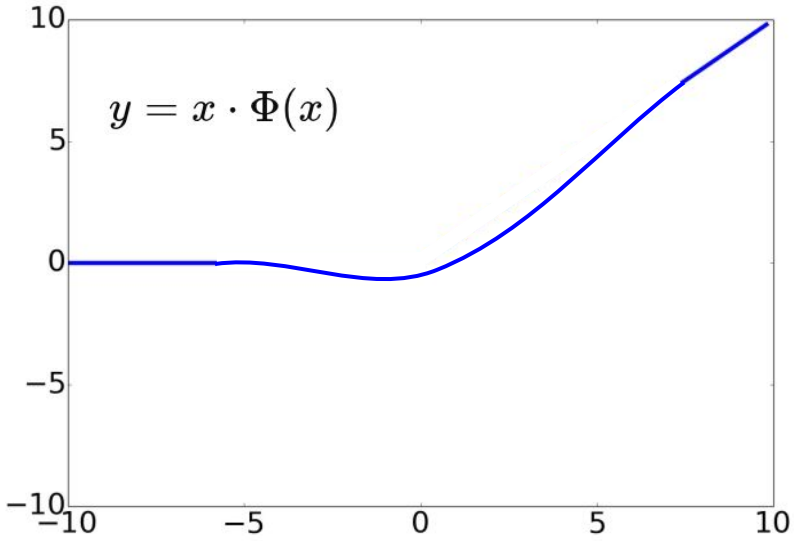


Tanh
Hyperbolic Tangent



ReLU
Rectified Linear Unit

Other Non-linear Activation Functions



GeLU

Gaussian Error Linear Unit

Phi é a função de distribuição acumulada (CDF) da distribuição normal padrão

Other Non-linear Activation Functions

Sigmoid: Smoothly differentiable and range (0, 1) compressing extreme values toward the boundaries.

Tanh: Smoothly differentiable, range (-1, 1) and saturating for large-magnitude values.

Other Non-linear Activation Functions

ReLU: Effective due to its near-linear behavior for positive inputs, while introducing non-linearity by zeroing out negative values.

GeLU: Smooth, non-linear activation that weights inputs by their magnitude, often providing better performance in deep networks.

Perceptron

A simple neural network proposed by Frank Rosenblatt (1957):

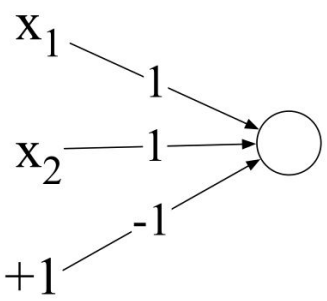
- Binary output (0 or 1);
- Linear activation function.

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

AND and OR with Perceptron

AND

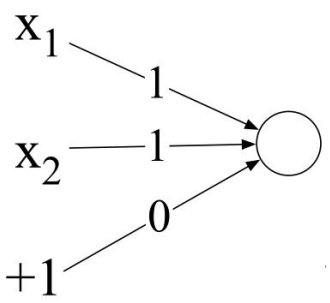
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1



AND

OR

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1



OR

The XOR problem

Can neural nets compute simple function of input?

XOR

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR with Perceptron

- Not possible to capture XOR with Perceptron;
- Perceptrons are linear classifiers:

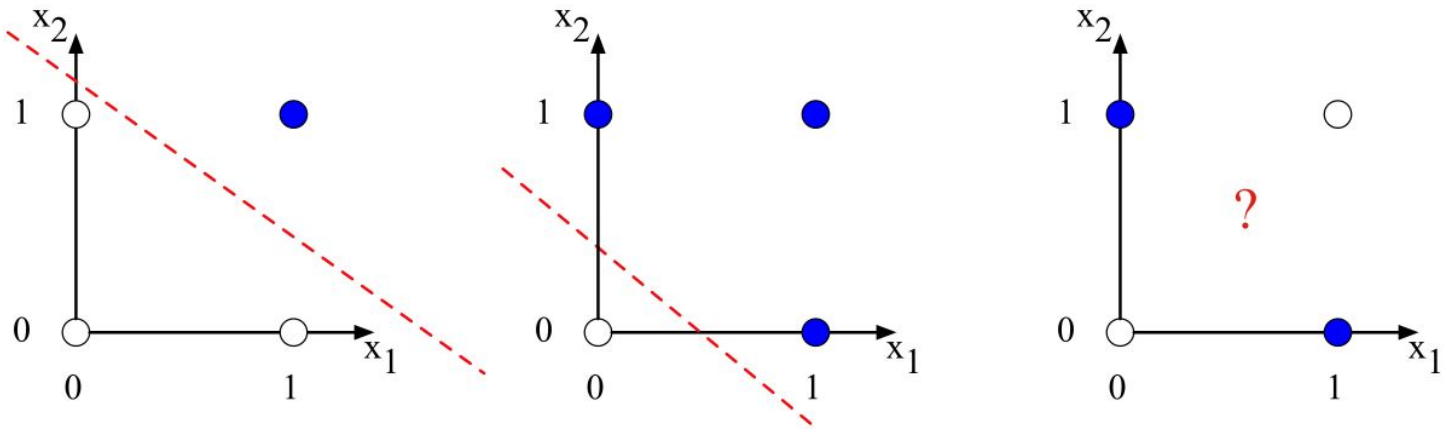
$$w_1x_1 + w_2x_2 + b = 0$$

- The lines acts as a decision boundary:
  - 0 if input is on one side of the line
  - 1 if on the other side of the line

Decision Boundaries

XOR is not a linearly separable function!

XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



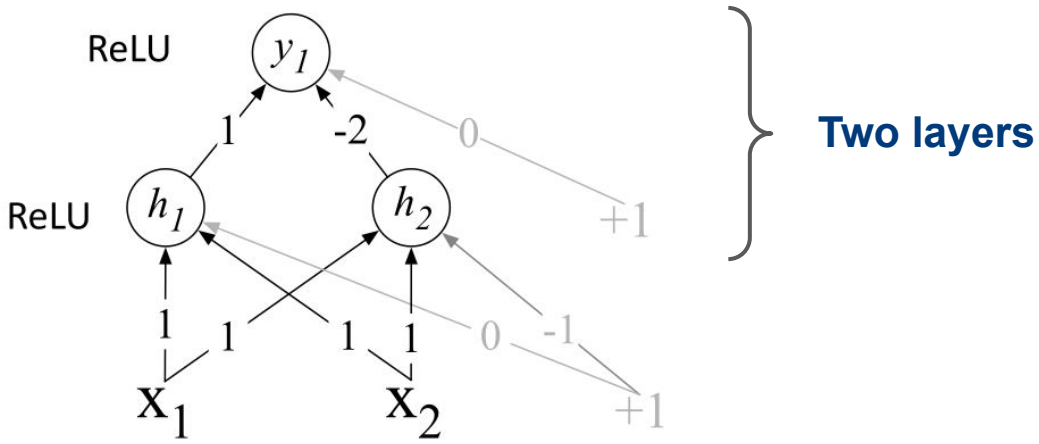
a) x_1 AND x_2 b) x_1 OR x_2 c) x_1 XOR x_2

Solution to the XOR problem

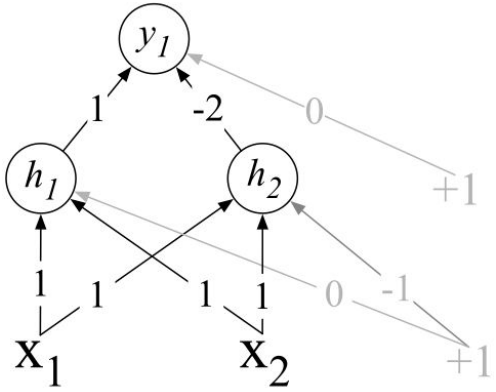
- XOR can not be calculates by a single perceptron;
- XOR can be calculated by a layered network of units.

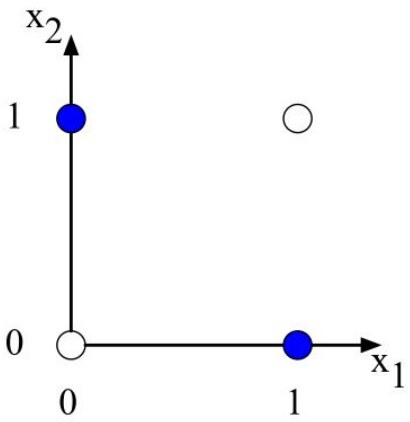
XOR

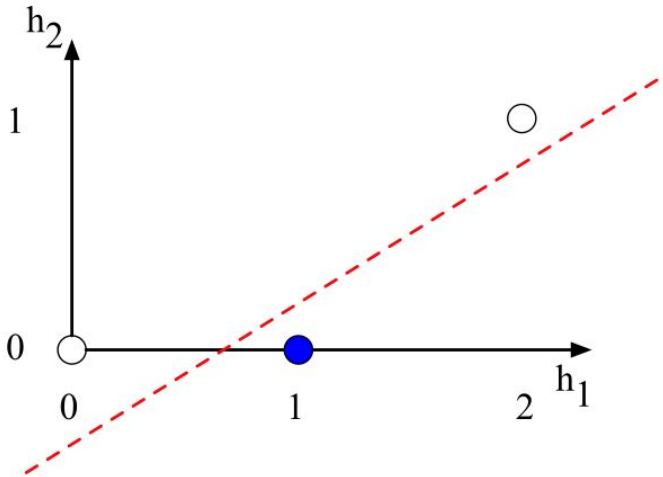
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



Solution to the XOR problem

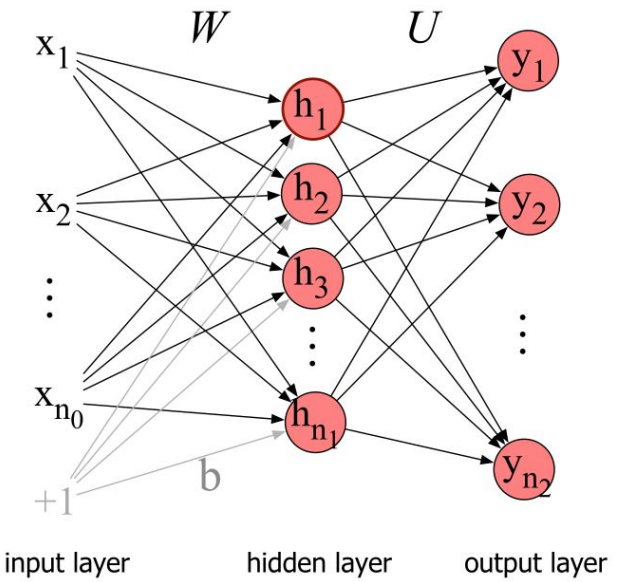






Hidden layers will learn to form useful representations

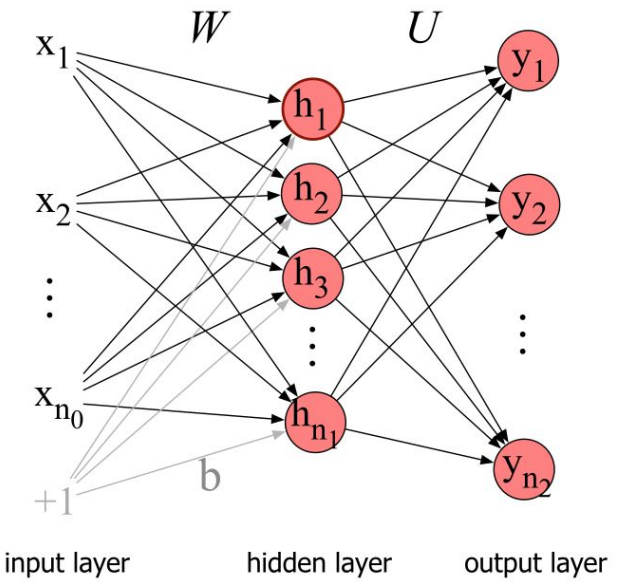
Feedforward Neural Networks



A simple 2-layer feedforward network, with one input layer, one hidden layer and one output layer

Two layers

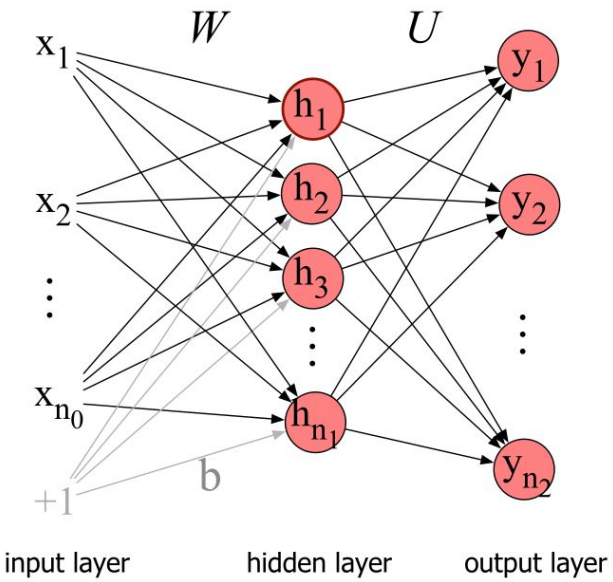
Feedforward Neural Networks



Two layers

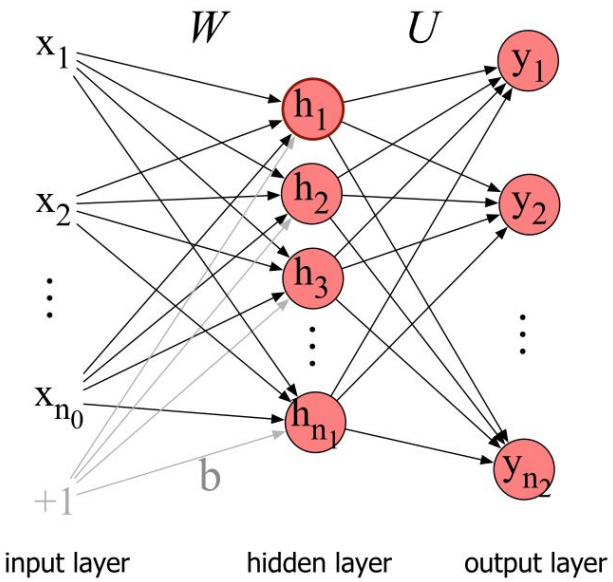
Input layer is layer 0
Hidden layer is layer 1
Output layer is layer 2

Feedforward Neural Networks



x is a vector of real numbers of dimension n_0

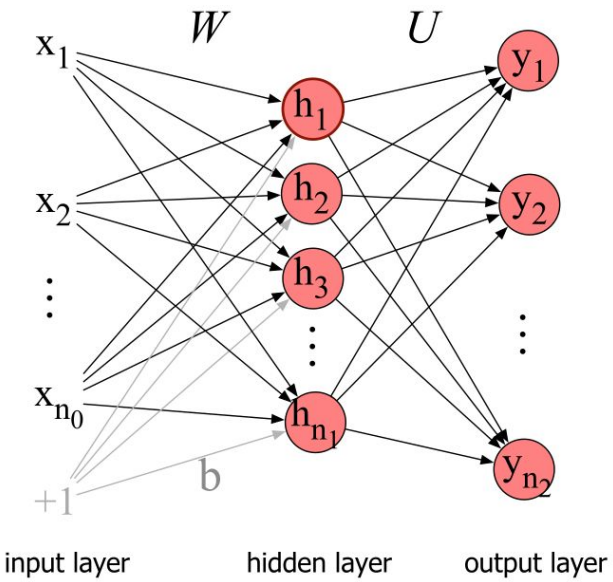
Feedforward Neural Networks



The hidden layer has dimensionality n_1

The output layer has dimensionality n_2

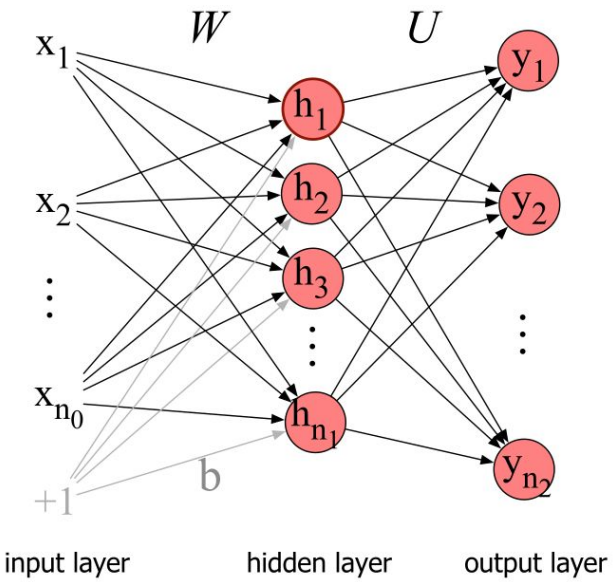
Feedforward Neural Networks



The weight matrix W has dimensionality W $[n_1 \times n_0]$

The weight matrix U has dimensionality U $[n_2 \times n_1]$

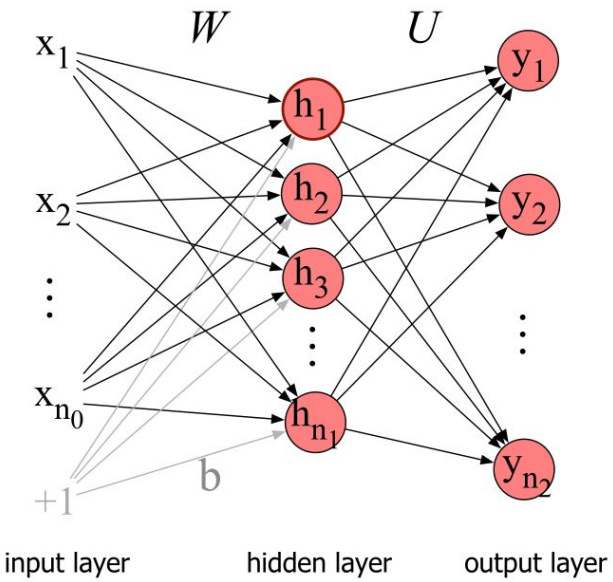
Feedforward Neural Networks



The output of the hidden layer, the vector h:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

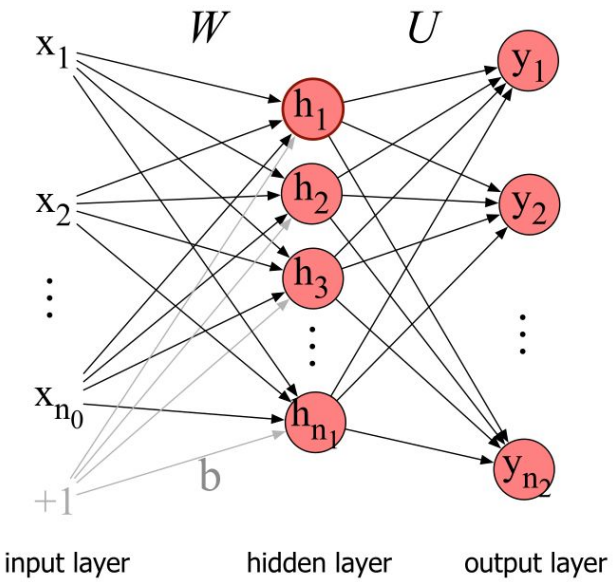
Feedforward Neural Networks



The output of the hidden layer for each h_j:

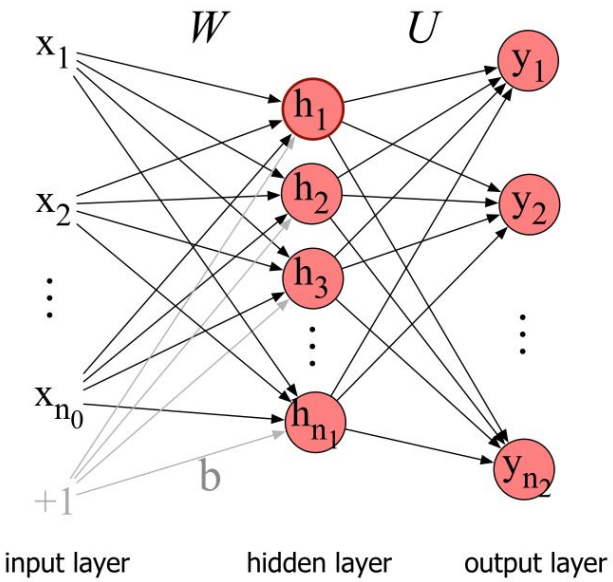
$$\sigma\left(\sum_{i=1}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i + \mathbf{b}_j\right)$$

Feedforward Neural Networks



Some models don't include a bias vector b in the output layer

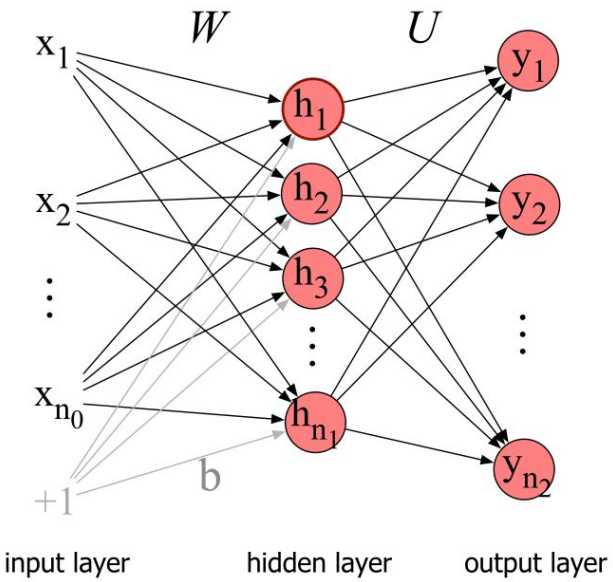
Feedforward Neural Networks



The weight matrix is multiplied by its input vector (h) to produce the intermediate output z:

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

Feedforward Neural Networks

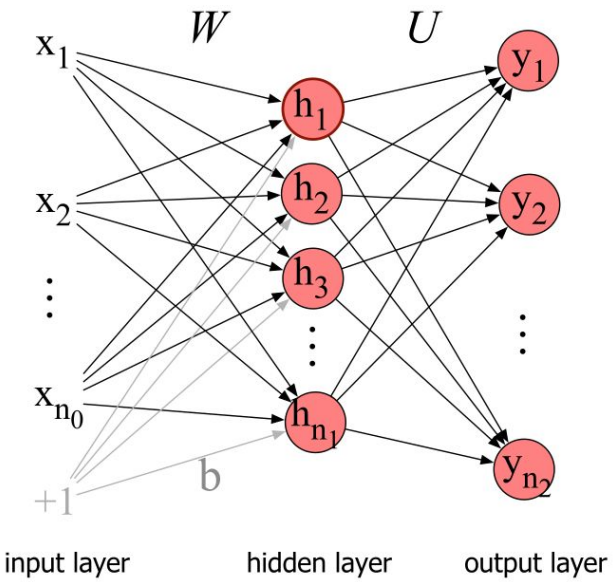


For a binary task, we might have a single output node.

The output y is the probability.

$$y = \sigma(z)$$

Feedforward Neural Networks

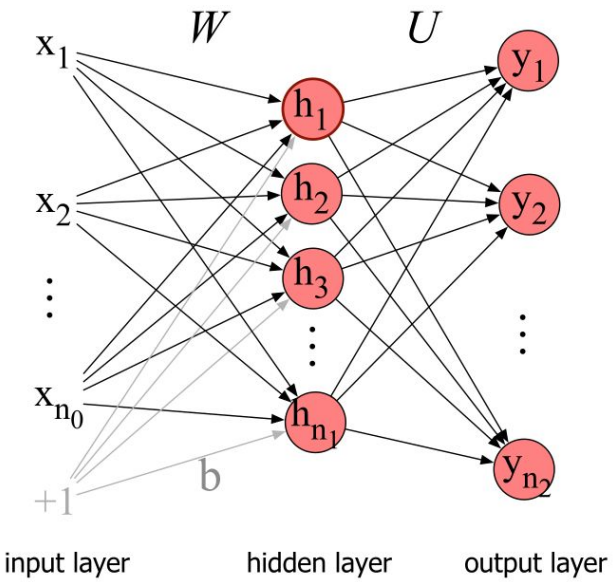


For a multinomial task, we must have one output node for each class

We need a vector of probabilities

$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^{d} \exp(\mathbf{z}_j)}$$

Feedforward Neural Networks



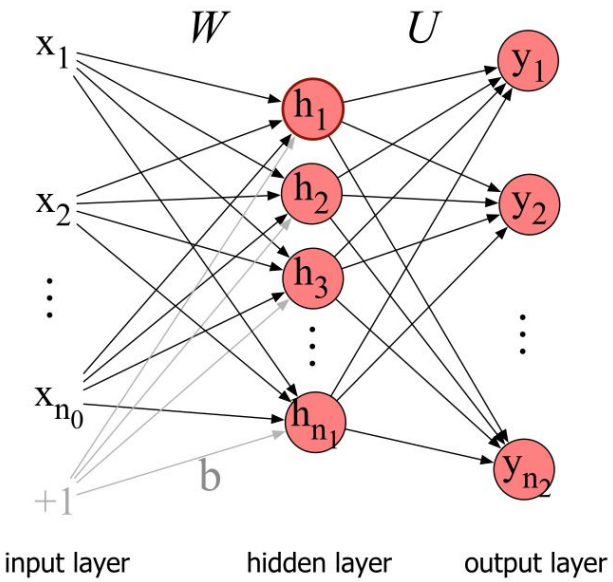
Summarizing:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\mathbf{y} = \text{softmax}(\mathbf{z}) \quad \textbf{or} \quad y = \sigma(z)$$

Feedforward Neural Networks



Summarizing:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{U}\mathbf{h}$$

$$\mathbf{y} = \text{softmax}(\mathbf{z}) \quad \textbf{or} \quad y = \sigma(z)$$

multiclass

binary

Softmax is a generalization of sigmoid

For a vector *z* of dimensionality *k*, the softmax is:

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right]$$

Example:

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

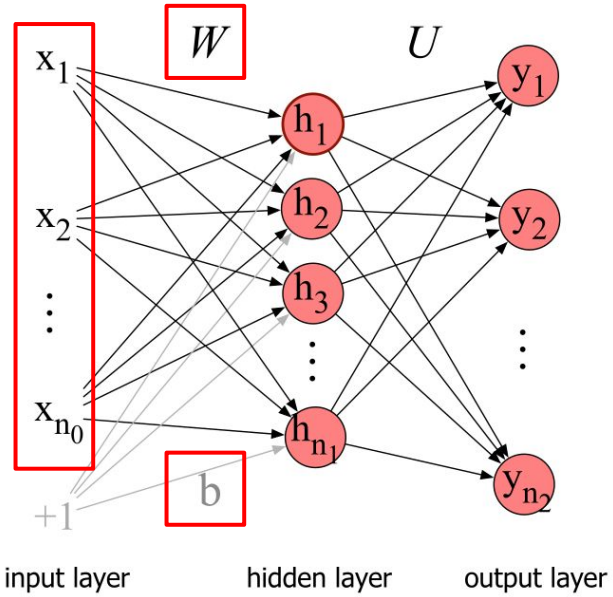
Notion

Superscript square brackets to mean layer numbers.

Example:

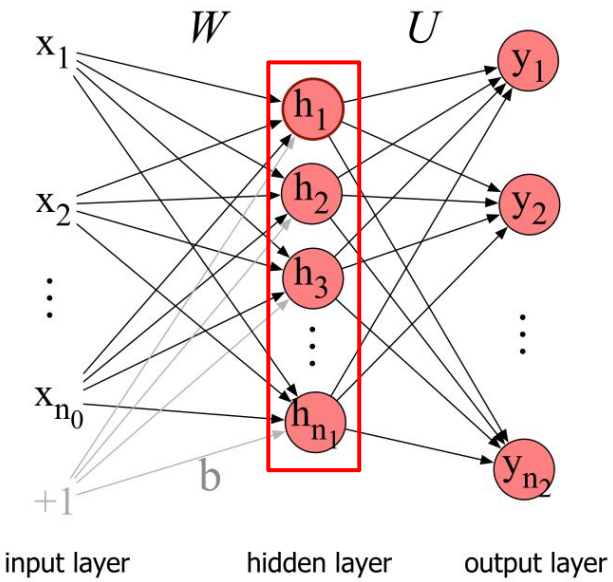
1. $W^{[1]}$ will mean the weight matrix for the (first) hidden layer
2. $b^{[1]}$ will mean the bias vector for the (first) hidden layer
3. n_j will mean the number of units at layer j
4. $g(\cdot)$ to stand for the activation function

Notion



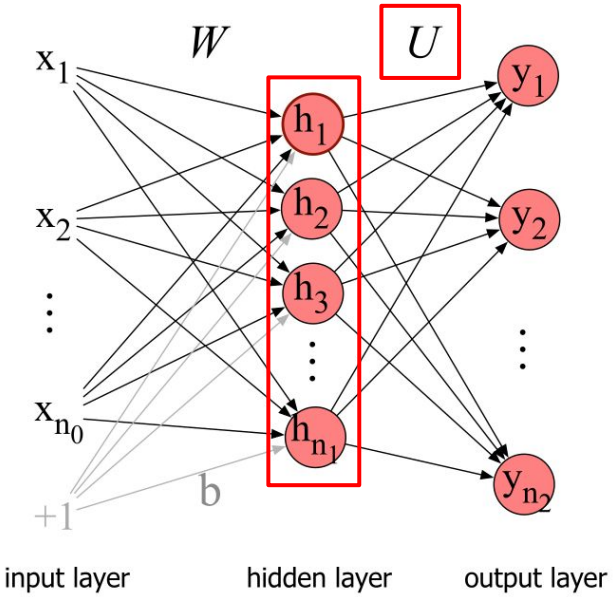
$$
\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} \\
\mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]} \\
\mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\
\hat{\mathbf{y}} &= \mathbf{a}^{[2]}
\end{aligned}
$$

Notion



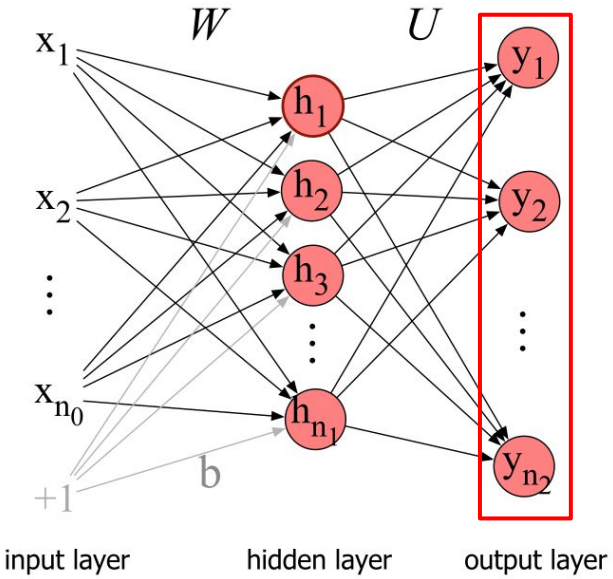
$$
\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} \\
\mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]} \\
\mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\
\hat{\mathbf{y}} &= \mathbf{a}^{[2]}
\end{aligned}
$$

Notion



$$
\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} \\
\mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} \\
\mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\
\hat{\mathbf{y}} &= \mathbf{a}^{[2]}
\end{aligned}
$$

Notion



$$
\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]} \\
\mathbf{a}^{[1]} &= g^{[1]}(\mathbf{z}^{[1]}) \\
\mathbf{z}^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} \\
\Rightarrow \quad \mathbf{a}^{[2]} &= g^{[2]}(\mathbf{z}^{[2]}) \\
\hat{\mathbf{y}} &= \mathbf{a}^{[2]}
\end{aligned}
$$

Replacing the bias unit

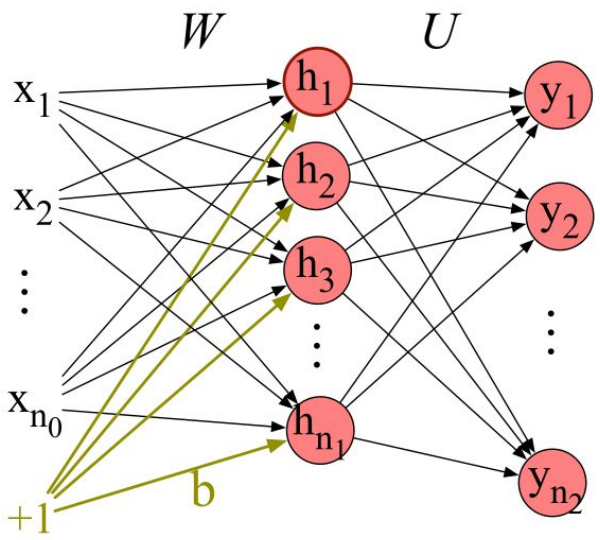
Switching to a notion without the bias unit.

Just a notational change:

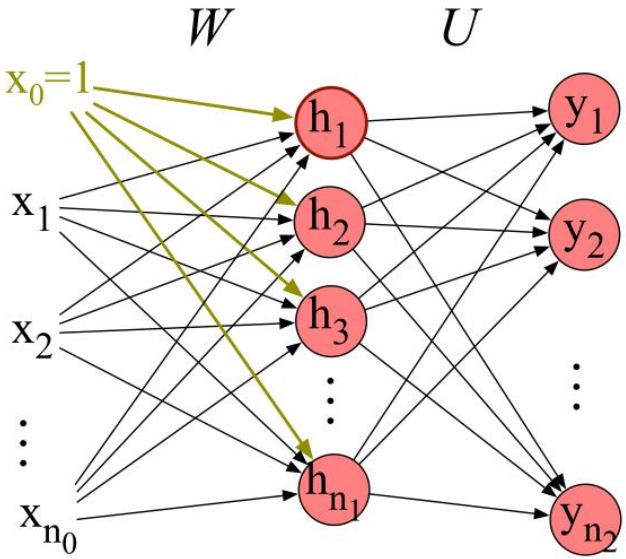
1. Add a dummy note a_0=1 to each leyer;
2. It is weight w_0 will be the bias
3. So the input layer $a^{[0]}_0 = 1$, $a^{[1]}_0 = 1$, $a^{[2]}_0 = 1$, …

Replacing the bias unit

Instead of:



Do this:



Replacing the bias unit

Instead of:

$$x = x_1, x_2, \ldots, x_{n0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma\left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j\right)$$

Do this:

$$x = x_0, x_1, x_2, \ldots, x_{n0}$$

$$h = \sigma(Wx)$$

$$h = \sigma\left(\sum_{i=0}^{n_0} W_{ji} x_i\right)$$

Use cases for feedforward networks

Let's consider 2 (simplified) sample tasks:

1. Text classification
2. Language modeling*

State of the art systems use more powerful neural architectures, but simple models are useful to consider!

Classification: Sentiment Analysis

Design the input layer as binary features

Classification: Sentiment Analysis

Design the input layer as binary features

Var	Definition
x_1	count(positive lexicon) $\in$ doc)
x_2	count(negative lexicon) $\in$ doc)
x_3	$\begin{cases} 1 & \text{if } \text{“no”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if } \text{“!”} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

Classification: Sentiment Analysis

Design the input layer as binary features

Adding a hidden layer allows the network to use non-linear interactions between feature

Classification: Sentiment Analysis

Design the input layer as binary features

Adding a hidden layer allows the network to use non-linear interactions between feature

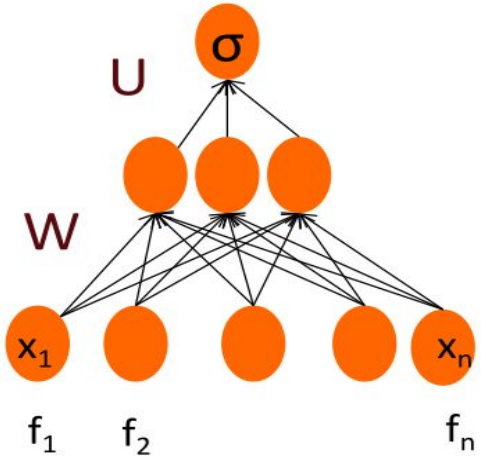
Output layer is Positive (1) or Negative (0)

Classification: Sentiment Analysis

Design the input layer as binary features

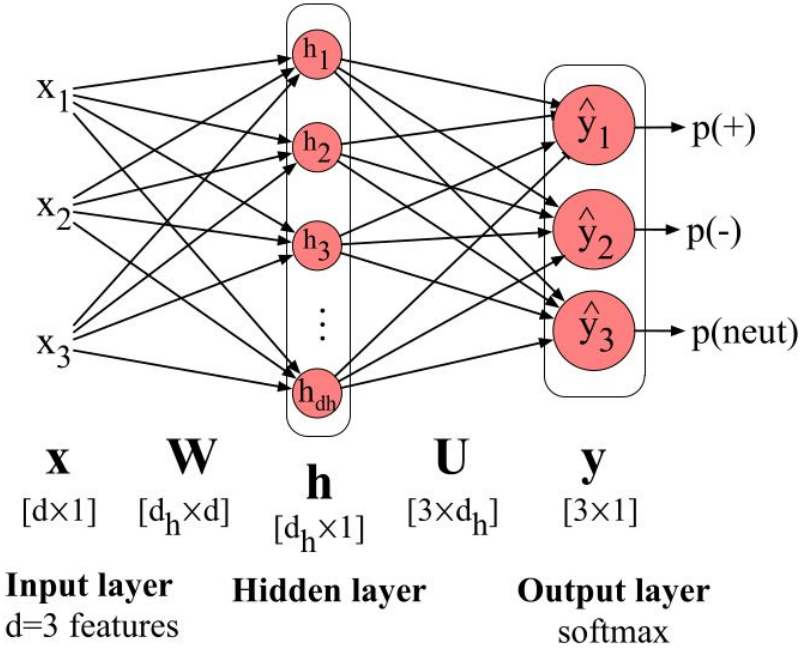
Adding a hidden layer allows the network to use non-linear interactions between feature

Output layer is Positive (1) or Negative (0)



Classification: Sentiment Analysis

Or a more sophisticated classifier:



Representation learning

- The real power of deep learning comes from the ability to learn features from the data;
- Instead of using hand-built human-engineered features for classification, use learned representations like embeddings!

Embedding matrix **E**

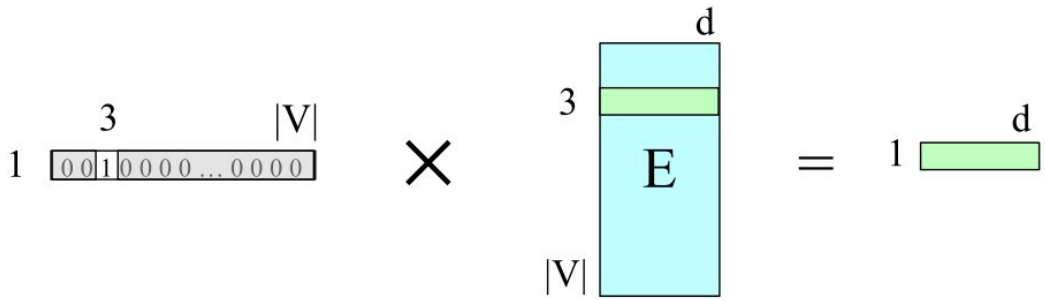
- An embedding is a vector of dimension [1 x d] that represents the input token.
- An embedding matrix **E** is a dictionary, one row per token of vocab V
- **E** has shape [|V| × d]

Text classification from embeddings

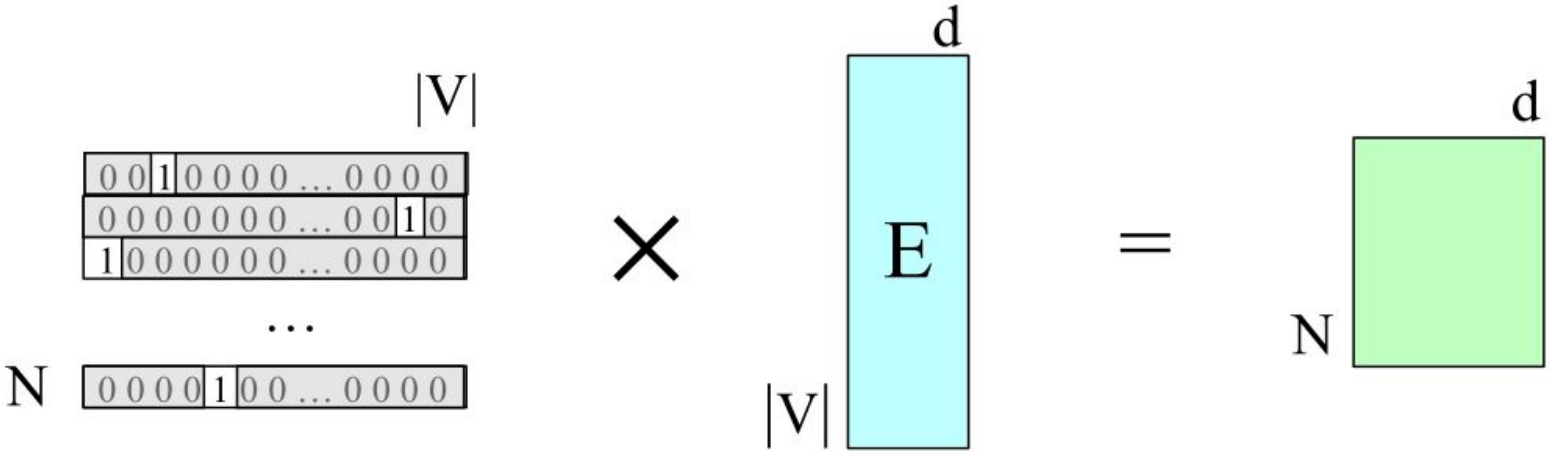
- Given tokenized input: `dessert was great`
- Select the embedding vectors from **E**:
  - Convert BPE tokens into vocabulary indices
  - w = [3, 9824, 226]
  - Use indexing to select the corresponding rows from **E**
  - row 3, row 4000, row 10532

Another way to think about **E**

- Treat each input word as one-hot vector
- If dessert is index 3: `[0 0 1 0 0 0 0 ... 0 0 0 0]`
  `1 2 3 4 5 6 7 ... ... |V|`
-
- Multiply it by **E** to select out the embedding for dessert



Collecting embeddings from **E** for **N** words



Text comes in different lengths

How to solve this issue:

- **Concatenate** all the inputs into one long vector of shape. If necessary, pad with zeros for shorter texts or truncate for longer texts
- **Pool** the inputs into a single short [1 × d] vector as sentence embedding (the same dimensionality as a word) to represent all the words. Less info, but very efficient and fast.

Pooling

Intuition: exact position not so important for sentiment.

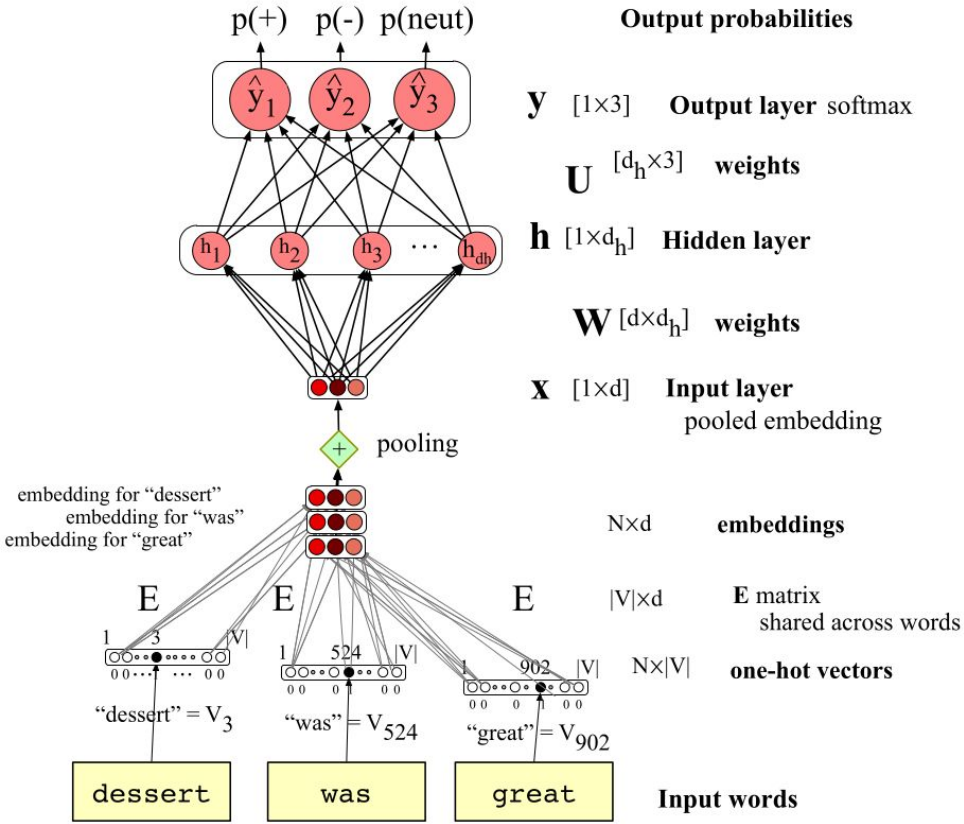
We'll just do some sort of averaging of all the vectors.

Mean pooling:

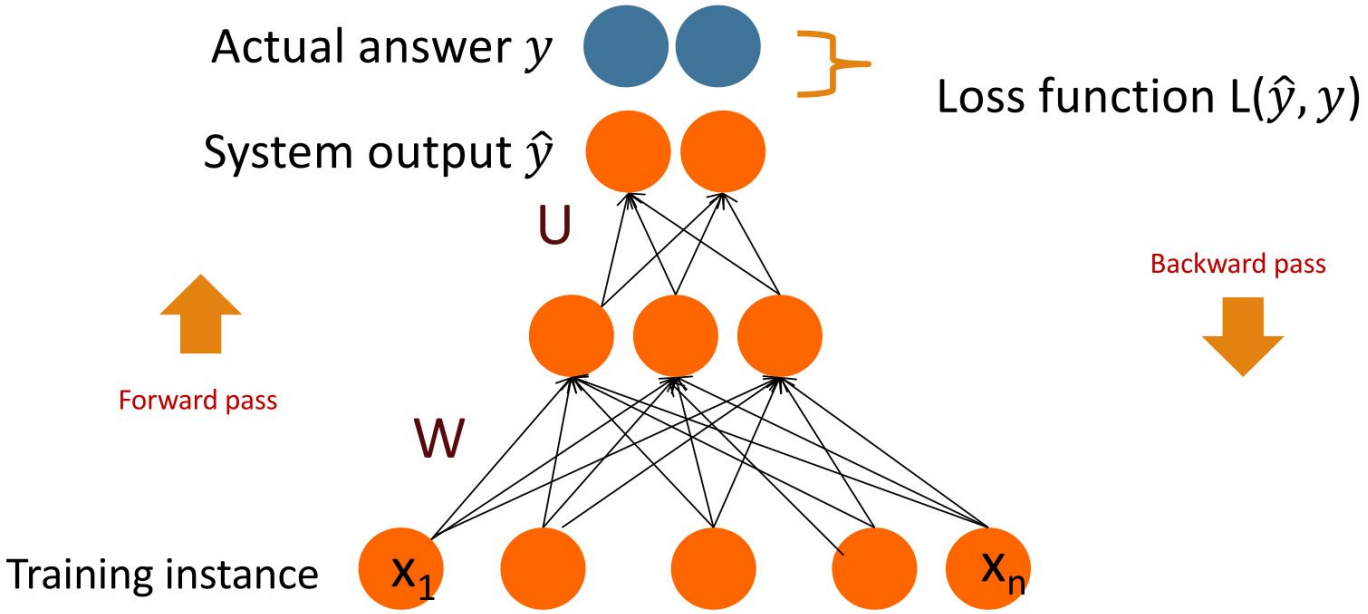
$$\mathbf{x}_{mean} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{e}(w_i)$$

$$\begin{aligned}
\mathbf{x} &= \text{mean}(\mathbf{e}(w_1), \mathbf{e}(w_2), \ldots, \mathbf{e}(w_n)) \\
\mathbf{h} &= \sigma(\mathbf{x}\mathbf{W} + \mathbf{b}) \\
\mathbf{z} &= \mathbf{h}\mathbf{U} \\
\hat{\mathbf{y}} &= \text{softmax}(\mathbf{z})
\end{aligned}$$

Pooling


p(+) p(-) p(neut)
Output probabilities
y [1×3] Output layer softmax
U [dh×3] weights
h [1×dh] Hidden layer
W [d×dh] weights
x [1×d] Input layer
pooled embedding
pooling
embedding for "dessert"
embedding for "was"
embedding for "great"
N×d embeddings
E
|V|×d E matrix
shared across words
N×|V| one-hot vectors
"dessert" = V3
"was" = V524
"great" = V902
dessert
was
great
Input words

Training Neural Nets



Training Neural Nets

For every training tuple (x,y)

- Run forward computation to find our estimate $\hat{y}$
- Run backward computation to update weights
- For every output node:
  - Compute loss between true label and the estimated label
  - Update the weight from hidden layer to the output layer
- For every hidden node:
  - Assess how much blame it deserves for the current answer
  - Update the weight from input layer to the hidden layer

Backpropagation Intuition

- Key question: Who is responsible for the error?
- Backpropagation idea:
  - Propagate the error backwards
  - Measure how much each weight contributed
  - Adjust weights to improve future predictions

The Chain Rule

- We want to compute how the loss changes with respect to each weight
- This involves:
  - Derivative of the loss
  - Derivative of the activation function
  - Derivative of the weighted sum

The Chain Rule

- We want to compute how the loss changes with respect to each weight
- This involves:
  - Derivative of the loss
  - Derivative of the activation function
  - Derivative of the weighted sum
- All combined using **Chain Rule**

Derivative of the Loss

Derivative of the activation function

Derivative of the weighted sum

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx} \quad \Rightarrow \quad \frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

Updating the weights

Use the derivative of the loss function with respect to weights

$$\frac{d}{dw} L(f(x;w), y)$$

The gradient is a vector of partial derivatives. To tell us how to adjust weights for each training item. Move them in the opposite direction of the gradient

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x;w), y)$$

Updating Hidden Layer Weights

Hidden units receive indirect error from the next layer

Each hidden neuron:

- Contributes to the output
- Receives a portion of the error (“blame”)

Updating Hidden Layer Weights

Derivative of the Loss

Derivative of the weighted sum (output layer)

Derivative of the activation function (hidden layer)

Derivative of the weighted sum (hidden layer)

$$\frac{\partial L}{\partial w^{[1]}} = \frac{\partial L}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial y^{[1]}} \cdot \frac{\partial y^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

Conclusion

- Neural networks learn by adjusting weights based on error
- Forward pass computes predictions
- Backward pass distributes error through the network
- Backpropagation:
  - Assigns responsibility to each weight
  - Uses the chain rule to compute gradients
- Gradient descent updates weights to reduce error

Conclusion

Key Takeaway:

- Neural networks learn by propagating error backward and adjusting weights to minimize it.

References

- JURAFSKY, Daniel; MARTIN, James H. Speech and language processing. 3. ed. Draft. 2026. Disponível em: https://web.stanford.edu/~jurafsky/slp3/